

Algoritmi e Strutture Dati – 2/7/2024

Esercizio A1 – Complessità – Punti ≥ 8

Utilizzando il Master Theorem, potremmo erroneamente concludere che la complessità è $\Theta(n\sqrt{n})$. Tuttavia, come richiesto dal compito, procediamo con il metodo della sostituzione per analizzare i limiti superiori e inferiori dell'equazione di ricorrenza data.

Limite inferiore È facile osservare che $T(n) = \Omega(n\sqrt{n})$, a causa del termine non ricorsivo dell'equazione di ricorrenza.

Limite superiore Utilizziamo lo stesso valore e proviamo a dimostrare che $T(n) \leq cn\sqrt{n}$ per qualche costante $c > 0$ e per tutti i valori $n \geq 1$.

- Casi base:

- $T(1) = 1 \leq c \cdot 1\sqrt{1}$, che è vera per $c \geq 1$;
- $T(2) = 1 \leq c \cdot 2\sqrt{2}$, che è vera per $c \geq \frac{1}{2\sqrt{2}}$;
- $T(3) = 1 \leq c \cdot 3\sqrt{3}$, che è vera per $c \geq \frac{1}{3\sqrt{3}}$;

- Ipotesi induttiva:

$$T(k) \leq ck\sqrt{k}, \forall k : 1 \leq k < n$$

- Sostituzione:

$$\begin{aligned} T(n) &= 7T(\lfloor n/4 \rfloor) + n\sqrt{n} \\ &\leq 7c \left(\frac{n}{4}\right) \sqrt{\frac{n}{4}} + n\sqrt{n} \\ &\leq \frac{7}{8}cn\sqrt{n} + n\sqrt{n} \\ &\leq cn\sqrt{n} \end{aligned}$$

L'ultima disequazione è vera se $c \geq 8$.

Conclusione Abbiamo così stabilito che $T(n) \leq cn\sqrt{n}$ per $c \geq 8$ e $n \geq 1$, e quindi $T(n) = \Theta(n\sqrt{n})$.

Esercizio A2 – Più o meno – Punti ≥ 10

La soluzione proposta esegue una visita in profondità dell'albero, utilizzando una funzione ricorsiva ausiliaria che restituisce una coppia di valori: la lunghezza del più lungo cammino *più-o-meno-k* nodo-foglia a partire dal nodo corrente e la lunghezza massima trovata nel sottoalbero. Se il cammino nodo-foglia si interrompe, restituiamo $-\infty$ per indicare la terminazione del cammino valido. In questo modo, tale valore non verrà selezionato dalla funzione `max()` quando l'altro valore è un intero.

Il funzionamento dell'algoritmo è il seguente:

- Se $t = \text{nil}$, restituiamo la coppia $(-\infty, -\infty)$, a indicare che non dobbiamo considerare questo ramo.
- Se t è una foglia, restituiamo $(0, 0)$, a indicare che abbiamo un cammino banale costituito da zero archi;
- Altrimenti, chiamiamo ricorsivamente la funzione a sinistra e a destra, ottenendo le coppie $(curr_R, max_R)$ e $(curr_L, max_L)$. Verifichiamo se il cammino può essere esteso, a sinistra come a destra (verificando prima che non sia interrotto in precedenza). A quel punto, scegliamo come valore `curr` il massimo fra i due valori `curr`; questo valore può assumere il valore $-\infty$ nel caso il percorso si sia interrotto da entrambi i lati. Selezioniamo il nuovo valore `max` come il massimo fra `max_L`, `max_R` e `curr`; in questo modo il valore può crescere se il cammino è stato esteso, altrimenti teniamo il valore massimo trovato in uno dei sottoalberi.

```
int longestPlusMinus(TREE T, int k)
```

```
    int, int max, val = PMRec(T, k)
    return max
```

```
int, int fontprocPMRec(TREE t, int k)
```

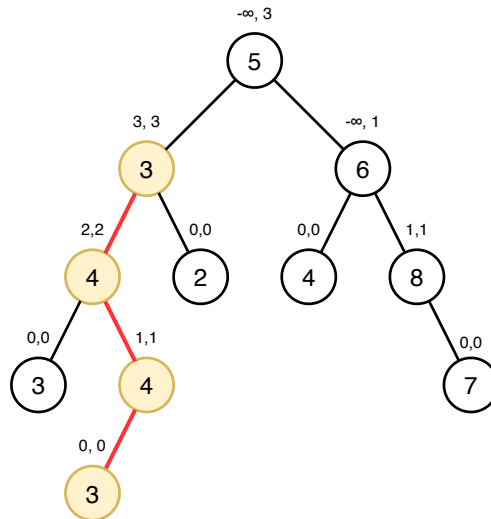
```

if t == nil then
    return (-∞, -∞)                                % Nodo nil
else if t.left == nil and t.right == nil then
    return (0, 0)                                    % Nodo foglia
else
    int, int curr_L, max_L PMRec(t.left, k)
    int, int curr_R, max_R PMRec(t.right, k)
    if curr_L ≠ ∞ and |t.key - t.left.key| ≤ k then
        curr_L = curr_L + 1                          % Il nodo t continua il cammino sul ramo sinistro
    else
        curr_L = -∞                                    % Il cammino sul lato sinistro si è interrotto
    if curr_R ≠ ∞ and |t.key - t.right.key| ≤ k then
        curr_R = curr_R + 1                          % Il nodo t continua il cammino sul ramo destro
    else
        curr_R = -∞                                    % Il cammino sul lato destro si è interrotto
    int curr = max(curr_L, curr_R)
    int max = max(max_L, max_R, curr)
    return (curr, max)

```

La complessità è $\Theta(n)$, perché si effettua una visita in profondità che esplora tutto l'albero.

È possibile osservare l'esecuzione dell'algoritmo nell'albero di esempio, con i valori che vengono restituiti dalla chiamata mostrati sopra i nodi.



Note sulla correzione Nel problema originale, il cammino di esempio veniva misurato come numero di nodi (4), mentre siamo abituati a contare il numero di archi. Nella versione che trovate qui contiamo correttamente il numero di archi e l'esempio è stato riformulato. Per adattare l'algoritmo alla versione originale, basta sostituire la coppia (0, 0) con la coppia (1, 1).

Molti hanno ignorato il fatto che il cammino da restituire dovesse essere nodo-foglia, e hanno scritto un algoritmo che restituisce la lunghezza del più lungo cammino nodo-nodo. In altre parole, hanno scritto un codice simile al seguente, in cui quando si incontra un nodo che non rispetta la regola, si mette 0 al valore del cammino corrente e si ricomincia a contare da quel nodo, ottenendo un cammino nodo-nodo. Due possibilità: o non avete letto il testo, o non vi siete resi conto che azzerando il valore potevate misurare anche altri cammini. Ho valutato 70% queste soluzioni, nonostante non siano corretti, perché l'errore è stato fatto da molti e le modifiche necessarie per rendere il codice corretto non sono molte.

```

int, int fontprocPMRec(TREE t, int k)


---


if t == nil then
    | return  $(-\infty, -\infty)$                                 % Nodo nil
else if t.left == nil and t.right == nil then
    | return (0, 0)                                          % Nodo foglia
else
    | int, int currL, maxLPMRec(t.left, k)
    | int, int currR, maxRPMRec(t.right, k)
    | if currL ≠ ∞ and  $|t.key - t.left.key| \leq k$  then
    | | currL = currL + 1                                % Il nodo t continua il cammino sul ramo sinistro
    | else
    | | currL = 0                                          % Il cammino sul lato sinistro si è interrotto, ripartiamo da qui
    | if currR ≠ ∞ and  $|t.key - t.right.key| \leq k$  then
    | | currR = currR + 1                                % Il nodo t continua il cammino sul ramo destro
    | else
    | | currR = 0                                          % Il cammino sul lato destro si è interrotto, ripartiamo da qui
    | int curr = max(currL, currR)
    | int max = max(maxL, maxR, curr)
    | return (curr, max)

```

Esercizio A3 – Circuiti matriciali – Punti ≥ 12

Per risolvere il problema presentato, possiamo adottare due strategie distinte. La prima strategia consiste nel costruire un grafo direttamente dalla matrice, connettendo ogni cella ai suoi vicini (due negli angoli, tre nei bordi e quattro nelle posizioni interne) e applicando un algoritmo per la ricerca di cicli. La seconda strategia, che scegliamo di seguire, adatta l'algoritmo per operare direttamente sulla matrice, semplificando la gestione dei dati senza la necessità di costruire un grafo intermedio.

Si noti che il grafo che deriva dalla matrice è un grafo orientato. Dobbiamo quindi ricordarci di non seguire gli archi all'indietro, altrimenti ogni coppia di caselle vuote è un ciclo. Era possibile desumere questo fatto dall'esempio che restituiva **false**. Qualcosa che non si intuiva dall'esempio, di proposito, è che una quadrato di dimensione 2×2 di caselle vuote è un circuito. Per la definizione, non è necessario verificare che contenga all'interno delle caselle piene.

Altra cosa importante è che l'insieme delle celle vuote può essere disgiunto, quindi il codice seguente prova a partire da ogni possibile cella che non sia già stata visitata.

```

boolean hasCircuit(boolean[][] M, int n)


---


    boolean[][] visited = new boolean[1...n][1...n] = {false}
    for r = 1 to n do
        | for c = 1 to n do
        | | if not visited[r][c] and M[r][c] = false then
        | | | if dfsRec(M, n, r, c, -1, -1) then
        | | | | return true
    | return false

```

La procedura ricorsiva è la seguente. L'unica cosa degna di nota sono le variabili $prev_r$ e $prev_c$, che rappresentano il nodo da cui si sta arrivando.

```
boolean dfsRec(boolean[][] M, int n, int r, int c, int prev_r, int prev_c)
```

```

visited[r][c] = true
int[] d_r = {-1, 1, 0, 0}
int[] d_c = {0, 0, -1, 1}
for k = 1 to 4 do
    int new_r = r + d_r[k]
    int new_c = c + d_c[k]
    if 1 ≤ new_r ≤ n and 1 ≤ new_c ≤ n and new_r ≠ prev_r and new_c ≠ prev_c then
        if visited[new_r][new_c] then
            return true
        else if not M[new_r][new_c] then
            if dfsRec(M, n, new_r, new_c, r, c) then
                return true
return false

```

La complessità è quella di una visita in profondità che deve toccare tutti i nodi, quindi $O(n^2)$.

Note sulla correzione In molti hanno dimenticato di escludere l'arco che torna sul nodo da cui si è appena arrivato. Ripassate l'algoritmo di ricerca cicli su grafi non orientati. In assenza di quel controllo, due qualunque caselle vicine vengono considerate un ciclo e viene restituito **true** anche per l'esempio di sinistra. Sono stato buono e ho dato una penalità di -10%. Se combinata con altri pasticci, tuttavia, il voto può essere inferiore.

Esercizio B1 – Vie di fuga – Punti ≥ 8

Il problema può essere risolto tramite una rete di flusso $G = (V, E, c, s, t)$.

L'insieme dei nodi V è dato da:

- Un nodo sorgente s ;
- Un nodo pozzo t ;
- Una coppia di nodi $in_{r,c}$ e $out_{r,c}$ per ogni posizione (r, c) tale che $1 \leq r \leq R$ e $1 \leq c \leq C$. La coppia serve a garantire che attraverso un posto possa passare un solo studente.

Il numero totale di nodi è $|V| = 2RC + 2$.

L'insieme degli archi E è dato da:

- Per ogni posizione (r, c) , un arco che connette $in_{r,c}$ con $out_{r,c}$, per un totale di RC archi.
- Per ogni posizione (r, c) tale che $1 \leq r \leq R$ e $1 \leq c \leq C$, aggiungiamo gli archi
 - $(out_{r,c}, in_{r+1,c})$ (alto, se $r < R$)
 - $(out_{r,c}, in_{r-1,c})$ (basso, se $r > 1$)
 - $(out_{r,c}, in_{r,c+1})$ (destra, se $c < C$)
 - $(out_{r,c}, in_{r,c-1})$ (sinistra, se $c > 1$),

per un totale di $4RC - 2R - 2C$ archi (ogni posizione della prima e ultima riga perde un arco verso l'alto/basso, ogni posizione della prima e ultima colonna perdono un arco sinistra e destra. Non ho bisogno di trattare diversamente gli angoli, che perdono archi in due direzioni, ma sono già contanti nella prima/ultima riga/colonna).

- Per ogni posizione (r_i, c_i) corrispondente all' i -esimo studente, aggiungiamo l'arco $(s, (in_{r_i, c_i}))$, per un totale di N archi).
- Per ogni posizione del bordo, aggiungiamo gli archi verso il pozzo:
 - $(out_{r,1}, t)$ (bordo sinistro, con $1 \leq r \leq R$)
 - $(out_{r,C}, t)$ (bordo destro, con $1 \leq r \leq R$)
 - $(out_{1,c}, t)$ (bordo alto, con $1 \leq c \leq C$)
 - $(out_{R,c}, t)$ (bordo basso, con $1 \leq c \leq C$)